

Teoría mínima de RL

Lo que hace falta saber para el laboratorio 1, y cómo está escrito en el código

Métodos numéricos · Matemáticas · lectura de 20 minutos · no hay que memorizar nada

1 La idea en una frase

Aprendizaje por refuerzo (*reinforcement learning*, RL): un programa toma decisiones, recibe puntos por sus consecuencias, y ajusta sus decisiones para ganar más puntos. Nadie le dice qué hacer; solo cuánto valió lo que hizo.

Es distinto de programar el método: en vez de escribir «corta en el medio», escribimos «te doy puntos por encoger el intervalo y te los quito por perder la raíz», y miramos si «cortar en el medio» *emerge*.

2 Los cinco ingredientes

nombre	qué es	en la bisección
agente	quien decide	el programa que no conoce el método
entorno	el mundo donde decide; responde a cada decisión con una observación y puntos	una función f continua y un intervalo $[a, b]$ con $f(a)f(b) < 0$
estado s	lo que el agente ve antes de decidir	los signos de $f(a)$, $f(x)$, $f(b)$
acción a	lo que puede hacer	la fracción λ donde cortar; qué lado conservar
recompensa r	los puntos que recibe justo después de actuar	$-1 + \log_2 \frac{\text{ancho anterior}}{\text{ancho nuevo}}$; -10 si pierde la raíz

Un **episodio** (en la interfaz, *generación*) es una partida completa: desde el intervalo inicial hasta que el ancho baja de la tolerancia, o hasta que se pierde la raíz. Se juegan miles de partidas, cada una con una f distinta.

Una **política** π es la regla del agente: en el estado s , hacer la acción $\pi(s)$. Al principio es aleatoria. El objetivo es encontrar la política que maximiza el **retorno**: la suma de recompensas del episodio,

$$G = r_1 + r_2 + \dots + r_n.$$

(En general se usa $G = \sum_k \gamma^k r_k$ con un factor de descuento $\gamma \in [0, 1]$ que da menos peso al futuro. En este laboratorio no hace falta: la bisección usa $\gamma = 0$, y la demostración $\gamma = 0,97$.)

Por qué el retorno de la bisección mide lo correcto

La recompensa de progreso *telescopica*: si w_0 es el ancho inicial y w_n el final,

$$\sum_{k=1}^n \left(-1 + \log_2 \frac{w_{k-1}}{w_k} \right) = \log_2 \frac{w_0}{w_n} - n.$$

El primer sumando está fijado por la tolerancia. Así que maximizar el retorno es exactamente **minimizar** n , el número de evaluaciones de f . La recompensa paso a paso solo sirve para que el agente vea progreso antes de terminar; no cambia qué es óptimo. (Esto se llama *shaping basado en potencial*, Ng, Harada y Russell, 1999.)

3 La tabla Q

El agente guarda una tabla con una fila por estado y una columna por acción. La entrada $Q(s, a)$ es su estimación de **cuántos puntos va a ganar en total** si en el estado s hace la acción a y después sigue actuando bien. Al principio toda la tabla vale 0.

	$\lambda = 0,1$	$\lambda = 0,2$	...	$\lambda = 0,5$	...		izq	der
						(+, -, +)	0,0	-10
						(+, +, -)	-10	0,0
						(-, +, +)	0,0	-10
						(-, -, +)	-10	0,0
estado «corte»	-0,55	-0,28		0,00				

Con la tabla aprendida, la política es leerla: en cada estado, la acción con mayor Q . Eso es la **política voraz** (*greedy*). En el informe, la tabla de la izquierda es «política de corte» y la de la derecha «regla de conservación».

Cómo se actualiza: Q-learning

Después de cada acción el agente tiene cuatro datos: estaba en s , hizo a , recibió r y llegó a s' . Entonces corrige la entrada:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{lo que ahora cree que vale}} - Q(s, a) \right].$$

Es decir: mueve la estimación vieja una fracción α hacia la nueva evidencia. La nueva evidencia es «lo que gané ahora» más «lo mejor que creo poder ganar desde donde caí». Con $\gamma = 0$ el segundo término desaparece y $Q(s, a)$ es simplemente el **promedio de las recompensas** obtenidas al hacer a en s ; con $\alpha = 1/N$ (siendo N las veces que se ha probado) ese promedio es exacto.

Ejemplo a mano. Estado «corte», acción $\lambda = 0,3$, $Q = 0$ al principio. Primera vez: la raíz cae en el 30 % izquierdo, se conserva ese lado, progreso $\log_2(1/0,3) = 1,74$, recompensa $r = 0,74$. Con $\alpha = 1/1$: $Q \leftarrow 0,74$. Segunda vez: la raíz cae a la derecha, progreso $\log_2(1/0,7) = 0,51$, $r = -0,49$. Con $\alpha = 1/2$: $Q \leftarrow 0,74 + \frac{1}{2}(-0,49 - 0,74) = 0,125$. Tras cientos de veces, $Q(0,3)$ converge a su promedio, unos $-0,12$. Y $Q(0,5)$ converge a 0, porque con $\lambda = 1/2$ el progreso es siempre exactamente 1 bit. Por eso gana.

Explorar o explotar

Si el agente siempre hiciera lo que la tabla dice mejor, nunca probaría las otras acciones y no descubriría que son mejores. Se resuelve con **ϵ -greedy**: con probabilidad ϵ hace una acción al azar (explora); si no, la mejor según la tabla (explota). ϵ empieza en 1 (todo al azar) y baja poco a poco; en el marcador de la interfaz se ve como « ϵ exploración». Además hay un **bono de**

curiosidad (UCB): a la hora de elegir, cada acción suma $c\sqrt{\ln N/(n_a + 1)}$, mayor cuanto menos se ha probado, para que las acciones poco visitadas parezcan atractivas hasta que se conozcan bien.

Esto explica dos cosas del laboratorio: por qué los hitos ocurren en generaciones distintas con otra semilla (la exploración es aleatoria), y por qué «la política voraz ya produce la demostración mínima» puede ocurrir *antes* de que el agente la produzca entrenando (mientras entrena, sigue explorando).

4 Dos agentes en la fase 1, y por qué

En la bisección hay dos decisiones por paso y se usan dos tablas independientes (dos «cabezas»):

- **cabeza de corte:** un solo estado y 9 acciones ($\lambda = 0,1, \dots, 0,9$). La fracción óptima no depende del ancho ni de la f , así que todas las muestras refuerzan la misma fila. Es un *bandido de 9 brazos*.
- **cabeza de conservación:** estado = signos de $(f(a), f(x), f(b))$, 2 acciones. Solo los signos deciden dónde está la raíz, y son lo único que el método real puede ver.

Un detalle de diseño: si se pierde la raíz, la culpa es de la conservación, no del corte, así que ese -10 solo actualiza la segunda tabla.

5 La fase 2: la demostración como juego

Mismo esquema, otro entorno. El **estado** es el conjunto de pasos ya establecidos (una lista de 21 casillas: 14 pasos verdaderos y 7 distractores). La **acción** es afirmar el siguiente paso. El entorno tiene un **verificador**: un paso es válido si todas sus dependencias ya están establecidas, no se ha usado, y no es distractor. **Recompensa**: -0,5 por paso válido (cada línea cuesta), -2 por paso inválido («salto lógico»), +20 al llegar a ■. Aquí sí importa el futuro ($\gamma = 0,97$): el valor del +20 se propaga hacia atrás por la cadena de dependencias, y por eso los pasos profundos entran en la política al final.

6 Cómo está escrito en el código

Todo lo anterior son unas 150 líneas en `01_biseccion/bisectrl/`. Estas son las que definen el juego.

Las funciones de prueba (`env_bisection.py`, `make_problem`)

En cada episodio se sortea una familia y unos parámetros. Todas tienen cambio de signo en $[a, b]$ y el entorno calcula la raíz de referencia con una bisección exacta (que el agente nunca ve).

familia	fórmula	parámetros aleatorios
poly3	$(x - r)((x - p)^2 + q)$	$r \in [-1,5, 1,5]$, $p \in [-2, 2]$, $q \in [0, 2, 2]$; una sola raíz real
sin	$\sin(kx) - c$	$k \in [0, 7, 1, 6]$, $c \in [-0, 8, 0, 8]$
exp	$e^x - c$	$c \in [1, 5, 6]$
cos_mix	$\cos x - x$	solo el intervalo: $a \in [-1, 0, 3]$, $b \in [1, 2, 2, 5]$
cubic_root	$(x - r)^3 + 0,3(x - r)$	$r \in [-1, 1]$; casi plana cerca de la raíz

```
def make_problem(rng):
    kind = rng.choice(["poly3", "sin", "exp", "cos_mix", "cubic_root"])
    if kind == "poly3":
        r = rng.uniform(-1.5, 1.5)
        p, q = rng.uniform(-2, 2), rng.uniform(0.2, 2)
        f = lambda x, r=r, p=p, q=q: (x - r) * ((x - p) ** 2 + q)
        a, b = r - rng.uniform(0.5, 2.5), r + rng.uniform(0.5, 2.5)
        ...
    if f(a) * f(b) >= 0:          # por seguridad numérica
        return make_problem(rng)
    root = _bisect_reference(f, a, b)
    return Problem(name, f, a, b, root)
```

Para añadir una familia basta con otro `elif` que defina `f`, `a` y `b`. Es un buen primer cambio que hacer.

El estado: qué ve el agente

```
def state_split(self):
    # un unico estado: la fraccion optima no depende del ancho
    return ("split",)

def state_keep(self):
    # solo los signos: es lo unico que decide que lado conserva la raiz
    sa = 1 if self.fa > 0 else -1
    sx = 1 if self.fx > 0 else (-1 if self.fx < 0 else 0)
    sb = 1 if self.fb > 0 else -1
    return ("keep", sa, sx, sb)
```

Fíjate en lo que **no** está en el estado: ni el valor de f , ni el ancho, ni la raíz. Si el estado contuviera la raíz, el agente «aprendería» a mirarla. La regla es: solo cosas que el método real puede medir.

La recompensa: qué le da puntos

```
REWARD_STEP = -1.0      # cada evaluacion de f cuesta
REWARD_LOST_ROOT = -10.0 # conservar el lado sin cambio de signo

def step_keep(self, keep):
    old_width = self.b - self.a
    ... # actualiza [a, b] al lado elegido
    if self.fa * self.fb > 0:
        return self.state_split(), REWARD_LOST_ROOT, True, "lost"
    progress = math.log2(old_width / (self.b - self.a))
    if self.b - self.a < self.tol:
        return self.state_split(), REWARD_STEP + progress, True, "converged"
    return self.state_split(), REWARD_STEP + progress, False, "step"
```

Cada return devuelve cuatro cosas: el nuevo estado, la recompensa, si el episodio terminó, y una etiqueta para la interfaz. En el laboratorio cambiarás REWARD_LOST_ROOT a 0 y verás qué pasa. La recompensa es la especificación completa del problema: si está mal, el agente aprende otra cosa, y lo aprende bien.

El agente: la tabla y su actualización (agent.py)

```
def act(self, state, actions=None):
    if self.rng.random() < self.eps:      # explorar
        return self.rng.choice(actions)
    if self.ucb_c > 0:                    # bono de curiosidad
        qs, ns = self.q[state], self.counts[state]
        n_state = sum(ns) + 1
        score = {a: qs[a] + self.ucb_c * math.sqrt(math.log(n_state) / (ns[a] + 1))
        for a in actions}
        return max(actions, key=score.get)
    return self.greedy(state, actions)    # explotar

def learn(self, s, a, r, s2, done):
    target = r if done else r + self.gamma * max(self.q[s2])
    self.counts[s][a] += 1
    alpha = self.alpha if self.alpha is not None else 1.0 / self.counts[s][a]
    self.q[s][a] += alpha * (target - self.q[s][a])
```

La línea de learn que empieza por `self.q[s][a] +=` es exactamente la fórmula de la sección 3. Es todo el «aprendizaje».

El verificador de la demostración (proof_kb.py, env_proof.py)

```
Step("ROOT", "Raíz",
     "El límite de una sucesión  $\leq 0$  es  $\leq 0$ :  $f(c)^2 \leq 0$ , así que  $f(c) = 0$ .",
     deps=("INV", "CONT"))

def is_valid(self, action, mask=None):
    step = STEPS[action]
    if step.distractor or (mask >> action) & 1:  # distractor o repetido
        return False
    return (DEPS_MASK[action] & mask) == DEPS_MASK[action]  # todas las deps
                                                                establecidas
```

El estado es un entero cuyos bits dicen qué pasos están establecidos; `is_valid` comprueba con una operación de bits que todas las dependencias están. Eso es todo lo que el «verificador» sabe de matemáticas: **lo que dicen las deps**. Si en ROOT quitas "INV" de deps, el verificador acep-

ta con gusto una demostración que no prueba el invariante. Ese es el experimento grafo del laboratorio.

7 Vocabulario para la sesión

episodio / generación	una partida completa, con una f nueva
retorno	suma de recompensas del episodio
política	regla estado $\rightarrow$ acción; voraz = leer el máximo de la tabla Q
$Q(s, a)$	estimación del retorno futuro tras hacer a en s
α	tasa de aprendizaje: cuánto se mueve Q hacia la nueva evidencia
γ	descuento: cuánto pesa el futuro (0: nada; 1: igual que el presente)
ε	probabilidad de explorar al azar
shaping	recompensa intermedia que mide progreso sin cambiar qué es óptimo
distractor	paso de la demostración que nunca es válido: falso, irrelevante o non sequitur
salto lógico	afirmar un paso cuyas dependencias no están establecidas

Para leer más: Sutton y Barto, *Reinforcement Learning: An Introduction* (2.^a ed., 2018), capítulos 2 (bandidos) y 6 (Q-learning). Gratis en línea. Los capítulos son más fáciles de lo que parecen.